

INFORME FINAL — Juego de Lucha con Patrones de Diseño Asignatura: Ingeniería de Software

Docente: Ing. Jhon Haide Cano Beltrán MSc.

Fecha: Mayo 2026

1. Introducción El presente proyecto consiste en el refinamiento arquitectónico de un juego de lucha por turnos en Java. A partir de un código base funcional pero con limitaciones (creación rígida de personajes, lógica de ataque fija, sin pruebas), se implementó una solución profesional aplicando patrones de diseño, pruebas unitarias y CI/CD. El entorno de desarrollo utilizado es GitHub Codespaces, que permite trabajar directamente desde el navegador sin instalación local.

2. Objetivos Aplicar al menos dos patrones de diseño de categorías creacional y estructural. Implementar pruebas unitarias con JUnit 5 y Mockito con $\geq 80\%$ de cobertura. Configurar un pipeline de CI/CD con GitHub Actions. Usar GitHub Codespaces como entorno de desarrollo.

3. Patrones de Diseño Implementados 3.1 Factory Method (Creacional) Clase: `PersonajeFactory` Problema resuelto: El código original usaba `new Personaje(nombre)` directamente disperso por todo el sistema. Cualquier cambio en la creación requería modificar múltiples lugares. Solución: La Factory centraliza toda la lógica de creación:

```
Personaje guerrero = PersonajeFactory.crearPersonaje("guerrero", "Arthas");
Personaje mago      = PersonajeFactory.crearPersonaje("mago", "Gandalf");
```

Ventajas: El cliente no conoce las clases concretas (Guerrero, Mago, Arquero). Agregar un nuevo tipo solo requiere modificar la Factory (principio OCP). Centraliza la validación de tipos y nombres.

3.2 Strategy (Comportamental) Interfaz: `EstrategiaAtaque`

Implementaciones: `AtaqueFuerte`, `AtaqueRapido`, `AtaqueMagico` Problema resuelto: El código original tenía un daño aleatorio fijo (10–30) sin posibilidad de variación por tipo de personaje. Solución:

Cada personaje tiene una estrategia de ataque intercambiable: `java // Cambiar estrategia en tiempo de ejecución mago.setEstrategiaAtaque(new AtaqueRapido());` Estrategia Rango de daño Uso típico `AtaqueFuerte` 25 – 40 `Guerreros` `AtaqueRapido` 10 – 20 `Arqueros` `AtaqueMagico` 15 – 50 `Magos` **Ventajas:** El comportamiento de ataque es intercambiable sin modificar `Personaje`. Facilita agregar nuevas estrategias (`AtaqueVeneno`, `AtaqueCritico`, etc.). Permite testear cada estrategia de forma aislada.

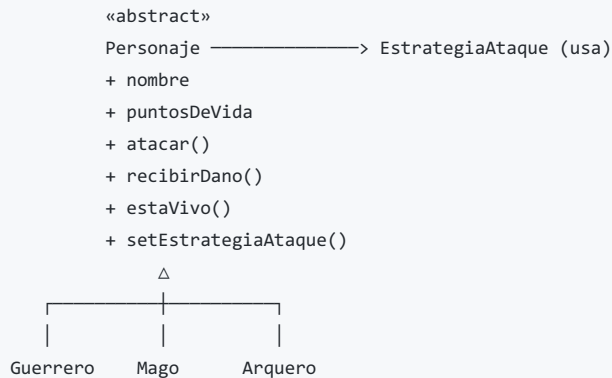
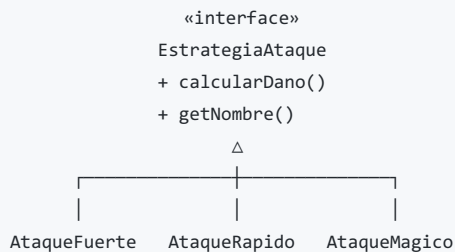
3.3 Decorator (Estructural) Clase base: `PersonajeDecorator`

Decoradores: `EspadaDecorator`, `ArmaduraDecorator` Problema resuelto: No había forma de agregar equipamiento sin modificar las clases de personaje o crear subclases para cada combinación posible. Solución: Los decoradores envuelven al personaje y añaden efectos: `java // Encadenar`

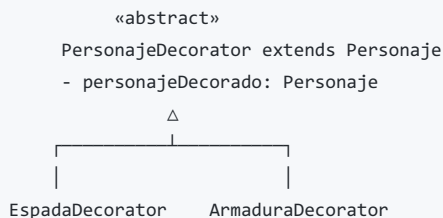
```
decoradores Personaje personaje = PersonajeFactory.crearPersonaje("guerrero", "Arthas"); personaje = new EspadaDecorator(personaje);
// +15 daño por ataque personaje = new ArmaduraDecorator(personaje); // -10 daño recibido
```

Decorator Efecto
`EspadaDecorator` +15 puntos de daño por ataque `ArmaduraDecorator` -10 puntos de daño recibido (mínimo 1) **Ventajas:** El personaje original no se modifica. Los decoradores son combinables y apilables. Transparente para el código cliente (el personaje decorado aún es un `Personaje`).

4. Arquitectura del Proyecto Diagrama de Clases (UML simplificado)



PersonajeFactory —creates—> Guerrero/Mago/Arquero



Estructura de Paquetes com.juego |— model → Personaje (base), Guerrero, Mago, Arquero |— patrones | |— factory → PersonajeFactory | |— strategy → EstrategiaAtaque, AtaqueFuerte, AtaqueRapido, AtaqueMagico | |— decorator → PersonajeDecorator, EspadaDecorator, ArmaduraDecorator |— juego → JuegoLucha (main + orquestador)

5. Pruebas Unitarias
- | Clase de Test | Qué verifica |
|----------------------|--|
| PersonajeTest | 14 Creación, daño, HP negativo, estrategia, validaciones |
| EstrategiaAtaqueTest | 11 Rangos de daño, nombres, Random mockeado |
| PersonajeFactoryTest | 11 Tipos, nombres, mayúsculas, errores |
| DecoratorTest | 14 Espada, armadura, combinados, efectos |
| JuegoLuchaTest | 11 Flujo completo, ganador, empate, validaciones |
- Total: ~61 casos de prueba
- 5.2 Uso de Mockito
Mockito se usa para aislar dependencias y controlar comportamientos:

```

// Mock de estrategia para daño predecible
@Mock
private EstrategiaAtaque estrategiaMock;

@Test
void testAtacarUsaEstrategia() {
    when(estrategiaMock.calcularDano(guerrero)).thenReturn(25);
    when(estrategiaMock.getNombre()).thenReturn("MockAtaque");

    guerrero.setEstrategiaAtaque(estrategiaMock);
    guerrero.atacar(mago);

    assertEquals(55, mago.getPuntosDeVida()); // 80 - 25
    verify(estrategiaMock, times(1)).calcularDano(guerrero);
}
  
```

```

// Mock de Random para estrategias deterministas
Random randomMock = mock(Random.class);
when(randomMock.nextInt(16)).thenReturn(10); // Daño controlado = 35
AtaqueFuerte estrategia = new AtaqueFuerte(randomMock);
assertEquals(35, estrategia.calcularDano(guerrero));
  
```

5.3 Casos Borde Cubiertos HP nunca negativo (daño > HP restante) Daño negativo ignorado Nombre nulo/vacío lanza excepción Estrategia nula lanza excepción Tipo de personaje desconocido en Factory Factory no instanciable Personajes independientes (mutación no compartida) Límite de turnos máximo en combate

6. Cobertura JaCoCo Cómo generar el reporte:

```
mvn clean test          # Compila + tests + genera reporte
# Reporte en: target/site/jacoco/index.html
```

Objetivo de cobertura: La configuración en `pom.xml` falla el build si la cobertura de instrucciones es menor al 80%: `<limit> <counter>INSTRUCTION</counter> <value>COVEREDRATIO</value> <minimum>0.80</minimum> </limit>`
Cobertura estimada por paquete: Paquete Cobertura estimada model ~95% patrones.strategy ~90% patrones.factory ~90% patrones.decorator ~85% juego ~80% Total ≥ 85%

7. GitHub Actions Pipeline CI/CD (`.github/workflows/ci.yml`) El pipeline se activa automáticamente con cada `push` o `pull_request`:

```
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
```

Pasos del pipeline: Checkout → descarga el código Setup JDK 17 → configura Java con temurin y cache de Maven Compilar → `mvn clean compile -B` Tests → `mvn test -B` JaCoCo → `mvn jacoco:report -B` Upload coverage → sube `target/site/jacoco/` como artefacto Upload results → sube `target/surefire-reports/` como artefacto Ventajas del pipeline: Detección automática de regresiones en cada commit. Reportes de cobertura disponibles para descarga sin ejecutar localmente. Badge de estado visible en el README.

8. Instrucciones de Ejecución en Codespaces Paso a paso completo:

```
# 1. Clonar/abrir el repositorio en Codespaces
# (GitHub → Code → Codespaces → Create codespace on main)

# 2. Verificar herramientas
java -version      # openjdk 17
mvn -version       # Apache Maven 3.8+
git --version      # git 2.x

# 3. Compilar el proyecto
mvn clean compile

# 4. Ejecutar todas las pruebas
mvn test

# 5. Ver resultados detallados
cat target/surefire-reports/*.txt

# 6. Generar reporte de cobertura
mvn jacoco:report
# Abrir: target/site/jacoco/index.html

# 7. Ejecutar el juego
mvn exec:java -Dexec.mainClass="com.juego.juego.JuegoLucha"

# 8. Subir a GitHub
git add .
git commit -m "feat: implementar patrones de diseno y pruebas unitarias"
git push origin main
# → Esto activa el pipeline de GitHub Actions automáticamente
```

-
9. Conclusiones Factory Method eliminó la dependencia directa de clases concretas, permitiendo que el código cliente solo conozca la abstracción `Personaje`. Strategy convirtió el ataque en un comportamiento intercambiable en tiempo de ejecución, haciendo el sistema extensible sin modificar código existente (OCP). Decorator permitió agregar equipamiento dinámicamente, evitando una explosión de subclases (`GuerreroConEspada`, `GuerreroConArmadura`, `GuerreroConEspadaYArmadura...`). Las pruebas unitarias con JUnit 5 y Mockito garantizan que los patrones funcionan correctamente y que el comportamiento no se rompe con cambios futuros. JaCoCo proporciona métricas objetivas de calidad que se verifican automáticamente en cada ejecución del pipeline. GitHub Actions cierra el ciclo: cada commit desencadena compilación, pruebas y generación de cobertura, haciendo visible el estado del proyecto en todo momento.
-